

Problem 3.10

```
class CourseGrading:
    def __init__(self):
        # Dictionary: student ID -> grade
        self.students = {}

        # Initialize 30 students with IDs 1 through 30
        for i in range(1, 31):
            self.students[i] = 0

    def change_grade(self, student_id, new_grade):
        if student_id in self.students:
            self.students[student_id] = new_grade
        else:
            print("Invalid student ID")

    def compute_gpa(self):
        total_points = 0

        for grade in self.students.values():
            if grade >= 90:
                total_points += 4.0
            elif grade >= 80:
                total_points += 3.0
            elif grade >= 70:
                total_points += 2.0
            elif grade >= 60:
                total_points += 1.0
            else:
                total_points += 0.0

        return total_points / 30

# Example usage
course = CourseGrading()
```

```

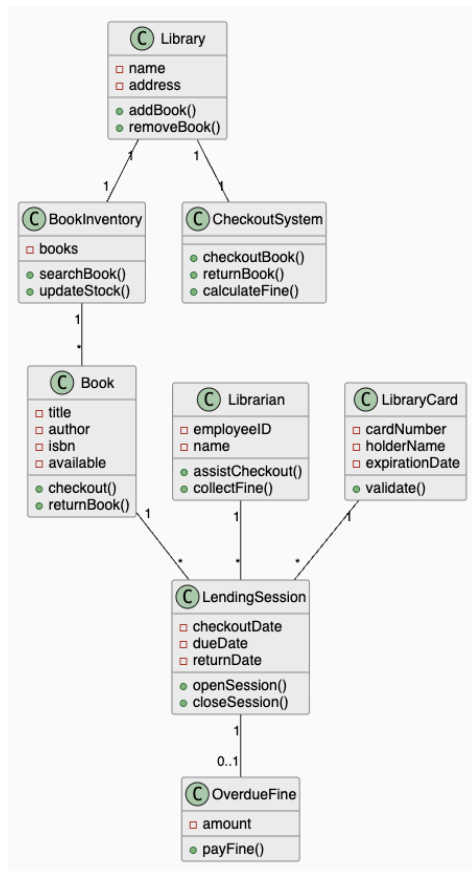
course.change_grade(1, 95)
course.change_grade(2, 82)
course.change_grade(3, 74)

print("Class GPA:", course.compute_gpa())

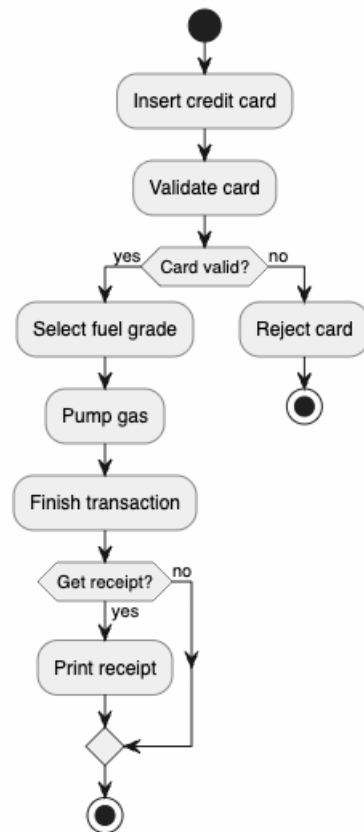
```

Problem 4.2

Using PlantUML (it's a web-based UML maker)



Problem 4.3



Problem 10.6

```

# Minimal Test Suite
assert program(40, 100) == 1 # First branch
assert program(70, 120) == 2 # Second branch
assert program(95, 200) == 3 # Else branch
assert program(20, 20) == 4 # Second independent if
assert program(10, 15) == 0 # No conditions true

```

```
print("All tests passed.")
```

Problem 10.8

```
class Grade:
    def getFinalGrade(self, score):
        if score >= 70:
            return "Pass"
        else:
            return "Fail"
```

Problem 10.8 Expanded

```
class Grade:
    def getFinalGrade(self, score):
        if score >= 90:
            return "A"
        elif score >= 80:
            return "B"
        elif score >= 70:
            return "C"
        elif score >= 60:
            return "D"
        else:
            return "F"

# Tests
testGrade = Grade()

assert testGrade.getFinalGrade(95) == "A"
assert testGrade.getFinalGrade(85) == "B"
assert testGrade.getFinalGrade(75) == "C"
assert testGrade.getFinalGrade(65) == "D"
assert testGrade.getFinalGrade(50) == "F"
```

Problem 11.2

Ambiguities

"Grilled" vs. "Broiled": The title describes "grilled" pockwester, but the final instruction says to "broil" it. In culinary and technical terms, these involve

different heat sources and equipment.

"Black Smoke": Using "clouds of black smoke" as a trigger to add ingredients is highly subjective and suggests the food is already ruined/burnt, making the timing impossible to replicate consistently.

"Soft to the touch": This is a vague metric for "doneness," especially for a fish-like creature, and does not provide a safe internal temperature or visual indicator.

Temperature Units: "170°" is mentioned without specifying Celsius or Fahrenheit.

Omissions

Missing Ingredients in List: The instructions call for mushrooms, sugar, and frobs, none of which appear in the initial ingredients list.

Unused Ingredients: The list includes garlic and eggplants, but these are never mentioned or used in the preparation steps.

Quantity Discrepancies: The instructions call for "2 cups of fresh orange juice," but the ingredients list only specifies "1 can of frozen orange juice." It is unclear if the user should reconstitute the juice or if the volume will match.

Preparation Details: There is no instruction on how to "dice" or "slice" the garlic (if it were used) or how to prepare the eggplant.

Contradictions

Lemon Preparation: The "night before" instructions say to squeeze and freeze the juice, but the following steps say to slice a lemon into "paper-thin slices" and add it to the mixture. It is unclear if this is the same lemon or a second one.

Orange Juice State: The ingredients list "frozen" juice, but the instructions demand "fresh" juice.

Egg Processing: The instructions state the eggs should have been "previously" whisked for 5 minutes, but the timing of when this "previous" action occurs relative to the caramelization of sugar is logically circular.

Vessel Conflict: The recipe starts in a skillet for the onions, but never explains

when or if the mixture moves to the saucepan where the wine and mushrooms are being prepared, yet they are eventually treated as one "mixture."

Flowchart Problem

I don't like installing things

Software Configuration Management Plan

Too much work for me

Software Test Plan or Test Procedure

https://docs.google.com/document/d/1eZqLtZiMH5NqHx2Kz1wK_KpuO5yPdh6DkfhAebhXpDQ/edit?usp=sharing